

PRODUCT REQUIREMENTS DOCUMENT

DocIngestQA v0.2.0

Pre-indexing QA auditor for RAG document ingestion pipelines.

Field	Value
Version	v0.2.0
Status	Shipped — PyPI live
Author	Sidharth Kriplani
Date	May 2026
Package	pip install docingestqa
License	MIT

1. Problem Statement

RAG pipelines fail silently at ingestion. Missing pages, OCR garble, duplicate chunks, and mojibake encoding errors all make it into the vector database undetected — then surface as hallucinations or missed retrievals in production. By the time you trace the answer failure back to a bad chunk, the issue has already been shipped.

The specific failures DocIngestQA targets:

Failure	Detection Method	RAG Impact
Missing pages from source	page_coverage vs. manifest	Entire page unindexed — hallucination on that content
OCR noise / replacement char	ocr_noise regex + ratio check	Garbled retrieval; LLM produces nonsense from noise
Exact/near-duplicate chunks	SHA-1 hash + Jaccard 5-gram	Retrieval bias; same content retrieved multiple times
Mojibake encoding corruption	encoding_health patterns	Unparseable tokens in embedding; retrieval failure
Sliding-window overlap artefacts	chunk_overlap 4-gram Jaccard	Redundant context; inflated context window cost
Mid-sentence split boundaries	split_quality heuristics	LLM receives incomplete evidence; drops context

2. Goals and Non-Goals

Goals:

- Run 11 deterministic quality checks on exported chunk sets before indexing.
- Produce structured JSON, Markdown, and HTML reports with chunk-level evidence.
- Integrate into CI/CD pipelines via CLI exit code 1 on FAIL status.
- Require zero non-stdlib dependencies for maximum deployability.
- Support AuditConfig for threshold customisation without code changes.

Non-Goals:

- Parsing documents from PDF/Word/HTML (that is the user's ingestion pipeline).
- Evaluating retrieval relevance or ranking quality.
- Verifying LLM answer faithfulness or hallucination rate.
- Cross-chunk semantic coherence or contradiction detection.
- Embedding quality assessment.

3. Check Specifications

Check	Version	WARN threshold	FAIL threshold	What it detects
input_summary	v0.1	—	0 chunks	Empty chunk set
metadata_completeness	v0.1	>5% missing	>20% missing	Missing source or page fields
document_coverage	v0.1	orphan sources	missing documents	Sources in manifest with no chunks
page_coverage	v0.1	—	any missing page	Pages declared in manifest absent from chunks
chunk_length	v0.1	>5% short/long	>10% empty	Chunks outside [80, 1500] char range
ocr_noise	v0.1	>5% noisy	>20% noisy	Replacement chars, junk runs, non-printable ratio
duplicate_chunks	v0.1	>=5 pairs	>=30 pairs	Exact (SHA-1) and near-dup (Jaccard 0.85 on 5-grams)
source_distribution	v0.1	1 source >80%	—	Single source dominance
chunk_overlap	v0.2	>=3 pairs	>=15 pairs	Consecutive chunks Jaccard>=0.40 on 4-grams
encoding_health	v0.2	mojibake>=5%	null bytes or mojibake>=20%	Null bytes, BOM, mojibake patterns
split_quality	v0.2	>=10% flagged	>=30% flagged	Mid-sentence starts/ends, navigation fragments

4. Failure Scenarios

Scenario A — PDF parser skips pages

A PDF with rotated pages causes the extraction library to silently skip pages 4 and 5. The chunk set has 60 chunks across 10 sources but 10 expected pages are absent. `page_coverage` FAILs. Without DocIngestQA, these pages are never indexed. Any query answered by that content produces a hallucination.

Acceptance criterion: Acceptance: `page_coverage` fires FAIL with `missing_page_total` ≥ 2 and correct source attribution.

Scenario B — Sliding-window splitter misconfigured

A developer sets `chunk_size=512` and `chunk_overlap=480` — nearly full overlap. Every adjacent chunk pair shares ~93% token content. `chunk_overlap` fires FAIL with `high_overlap_pair_count` $\gg 15$. The index will have massive redundancy and inflated storage cost.

Acceptance criterion: Acceptance: `chunk_overlap` fires FAIL with at least 15 high-overlap pairs flagged.

Scenario C — OCR pipeline on scanned document

A scanned invoice is OCR'd with a miscalibrated model. 8% of chunks contain replacement characters (U+FFFD) or strings like 'l0gin' (digit-for-letter). `ocr_noise` fires WARN. The embedding of a garbled chunk will map to a random location in embedding space, degrading retrieval precision.

Acceptance criterion: Acceptance: `ocr_noise` fires WARN with `noisy_chunk_count` > 0 and chunk-level evidence showing the pattern.

Scenario D — Duplicate content from template re-use

A knowledge base has 40 FAQ entries that all begin with the same 3-sentence boilerplate. `duplicate_chunks` fires WARN (5+ near-duplicate pairs at Jaccard > 0.85). These chunks will all retrieve together, consuming the context window and pushing relevant unique content out of the LLM's attention.

Acceptance criterion: Acceptance: `duplicate_chunks` detects ≥ 5 near-duplicate pairs and includes source + text preview in evidence.

Scenario E — Re-run idempotency

The same chunk set is audited twice. The second run must produce identical output (same check statuses, same issue counts, same severity). All checks are deterministic — no randomness, no side effects, no timestamp-dependent thresholds.

Acceptance criterion: Acceptance: two consecutive runs on identical input produce byte-identical JSON output (excluding `generated_at` timestamp).

5. Quantitative Acceptance Criteria

Criterion	Target	Check
page_coverage FAIL on any missing page	tolerance	page_coverage fires at missing_page_total >= 1
chunk_overlap FAIL threshold	>= 15 high-overlap pairs	consecutive Jaccard >= 0.70
encoding_health FAIL on null bytes	null_byte_count >= 1	immediate FAIL regardless of rate
duplicate_chunks FAIL threshold	>= 30 pairs	SHA-1 exact or Jaccard >= 0.85
split_quality FAIL threshold	>=30% chunks flagged	mid-sentence or nav fragment heuristics
CLI exit code	1 on FAIL, 0 on PASS/WARN	tested in test_checks.py
Output schema version	'0.2' in schema_version field	all output formats
Idempotency	identical JSON on re-run	excluding generated_at
Dependency count	0 non-stdlib at runtime	jinja2 optional for HTML

6. Roadmap

Version	Scope
v0.1.0	8 checks, JSON/MD output
v0.2.0	3 new checks (overlap, encoding, split quality), CLI, HTML output
v0.3	Configurable severity overrides; per-source report sections; chunk-type tagging
v1.0	Semantic coherence check (embedding cosine within chunk); LLM-narrated summary option